Hashing

Part 13

1

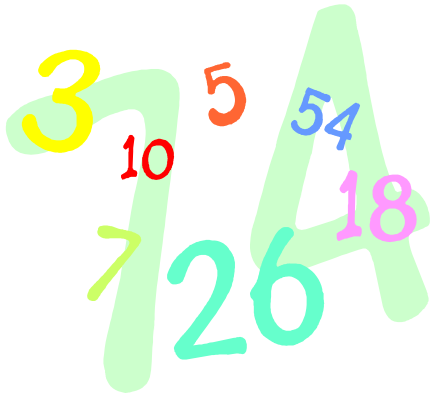
Hashing

We have a need… a need for speed!

2

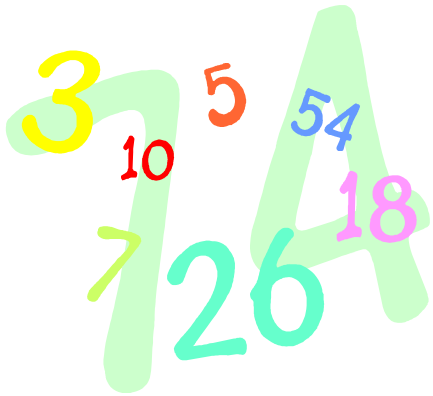
Hashing

- Array elements can be accessed in O(1)
- Why? The memory address of any element can be calculated mathematically
- … however, this doesn't work for dictionary keys



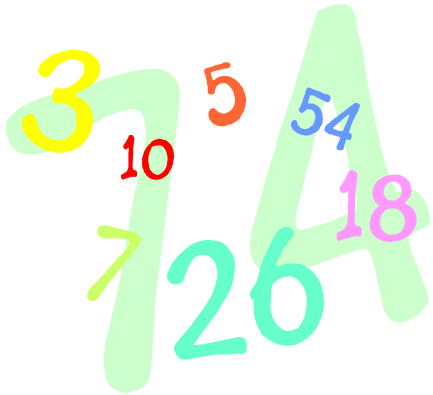
Hashing

- We can use a nice balanced tree to store the data
- … but that is O(log n) – which is still excellent
- Is it possible to get the time complexity down to O(1)?

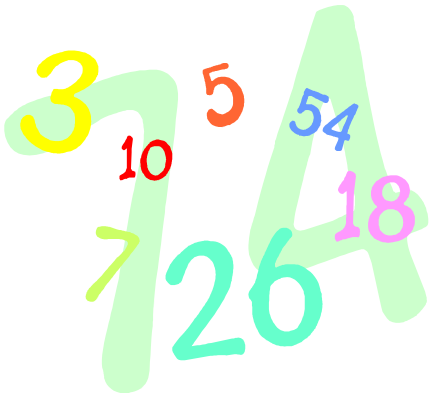


Hashing

- What if we came up with a *"magic function"* that converted keys into array indexes?
- A *hash function* takes a key object as an argument and returns a numeric index
- hash(key) → index



Hashing



- Given a specific key the hash function would compute the *exact* index of the element
- This will given dictionaries O(1) access

Hash Mathematics

- A hash function maps keys into indexes in a *hash table*
- For element e with key k and h is hash function…
  - e is stored in position $h(k)$ in the hash table *(an array)*
  - to find/store for e, compute $h(k)$ to locate position

Hash Functions

- A hash function can be *anything*
- However, it is best to...
  - find one that spreads items evenly over the array
  - ... and one that limits collisions

Using a Hash

Hash function

Key	Hash
cat	0
turkey	4
dog	1
possum	5

Hash Table

Index	Data
0	cat object
1	dog object
2	
3	
4	turkey object
5	possum object

Example Hash Function

- `hash("Rick") = 5`
- `hash("Morty") = 1`
- `hash("Jerry") = 0`
- `hash("Beth") = 4`

Array	
0	Jerry
1	Morty
2	
3	
4	Beth
5	Rick

But, there are Problems

- Simple hash functions
  - work for implementing dictionaries
  - but most apps have key ranges that are too large for 1-1 mapping between hashes and keys
- Example:
  - key range from 0 to 65,535
  - collection will have no more than 100 items at any given time
  - impractical to use a hash table with 65,536 slots!

Finding the Hash Function

- There is **no** magic function
  - **only** in rare cases, with a limited key range, a perfect function can exist
  - however, for real World cases, there is no function possible
- So, we can take a different approach
  - don't use the hash value as a finishing point
  - use it as a location to start looking

Collisions

- When two keys hash to the same array location, this is called a *collision*
- What do we do?
  - normally collisions are “first come, first serve”
  - the first key that hashes to the location gets it
  - so, we need to decide what do with the second item that hash to the same location
  - there are two solutions…

Example Hash Function

- `hash("Ash") = 5`
- `hash("Jesse") = 1`
- `hash("Brock") = 0`
- `hash("Misty") = 4`
- `hash("James") = 1`

Array	
0	Brock
1	COLLISION
2	
3	
4	Misty
5	Ash

Closed Hashing

Chaos... good news

Collision Solution: Closed Hashing

- With *closed hashing*, we use the existing array and search for an empty position
- Use the hash value as start position – we start searching here

Collision Solution: Closed Hashing

- If the array element is a occupied...search down and look for an empty element
- The search must also…
  - wrap-around to the top
  - and be aware if the search cycles through the entire array – *we ran out of space*

Closed Hash Example

- Let assume we are adding "Pacman" which has a hash value of **0**
- This collides with "Dig Dug"
- We search down for the next empty element

0	Dig Dug
1	Q-Bert
2	
3	
4	Frogger
5	

Closed Hash Example

- Now, at index **1**, we collide with Q-Bert
- We search down for the next empty element

0	Dig Dug	✖
1	Q-Bert	✖
2		
3		
4	Frogger	
5		

Closed Hash Example

- Finally, we find an empty array element
- Pacman is stored here
- Note, this is **2** positions off from the original hash

0	Dig Dug	✖
1	Q-Bert	✖
2	**Pacman**	
3		
4	Frogger	
5		

Closed Hashing Clustering

- One problem with the closed hashing is the tendency to form *clusters*
- A cluster is a group of continuous used array elements – with no open slots

0	Dig Dug
1	Q-Bert
2	Pacman
3	
4	Frogger
5	

Closed Hashing Clustering

- The bigger a cluster gets, the more likely it is that new keys will hash into it *(and collide)*
- This will cause the hash will eventually degrade to **O(n)**

0	Dig Dug
1	Q-Bert
2	Pacman
3	
4	Frogger
5	

Closed Hashing Pitfalls

- Closed hashing is not the best solution
- It requires a static array
  - the array cannot be increased at runtime (or the hash fails)
  - hence, the array is finite
- Clustering causes O(n) degradation

Closed Hashing Pitfalls

- You cannot delete items
  - it creates empty slots in clusters!
  - this can prevent an item, *added in a cluster*, from being found below the gap
  - there are work-rounds, but it gets **convoluted**

Closed Delete Problem

- "Dig Dug" and "Pacman" hash to **0**
- When "Pacman" was added it had to be stored at **2**
- This is a cluster

0	Dig Dug
1	Q-Bert
2	**Pacman**
3	
4	Frogger
5	

Closed Delete Problem

- If "Dig Dug" is deleted, the array element is empty
- If there is a search for "Pacman", it will hash to **0** and it <u>won't</u> be found

0	
1	Q-Bert
2	**Pacman**
3	
4	Frogger
5	

Open Hashing

The Merging of Concepts

27

Open Hashing

- With *open hashing*, we don't store individual objects in each array element
- Instead, each array element is a linked list or a tree (preferably balanced)

28

Open Hashing

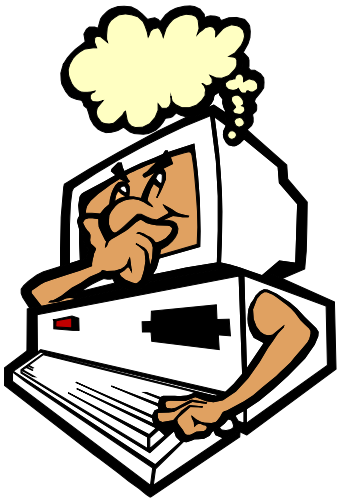
- So, our hash table is an array of either linked lists or trees
- This approach is also known as *bucket hashing* since each list/tree acts a "hash bucket"

Collision Solution: Open Hashing

- When a collision occurs the item is added to the list/tree
- So, this list/tree will contain **all** the objects with the same hash value
- We don't need to look for conflicts

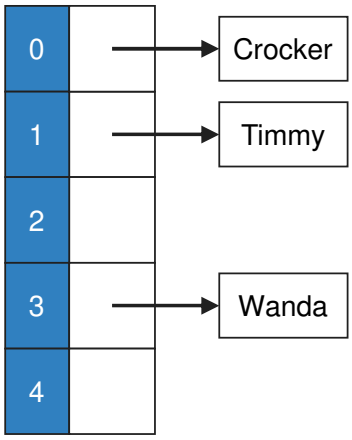
When an Object is Looked Up…

- Compute the hash value
- And then search the targeted list/tree
- For example, for a balanced tree, searching would be O(1) at best and O(log n) at worst



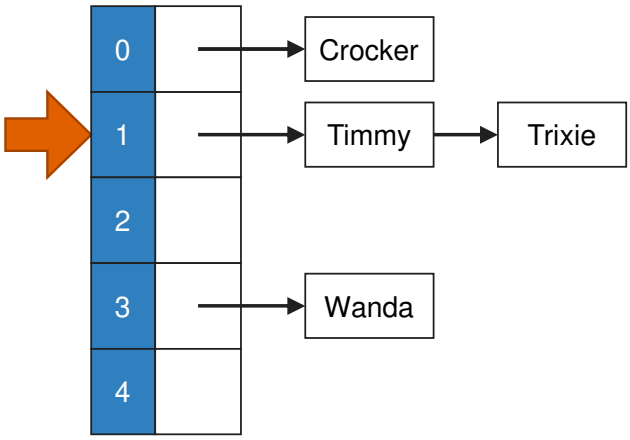
Open Hash Example

- Here we have a open hash table contain characters from Fairly Odd Parents
- Each table entry, in this case, contains a simple linked-list
- The head node is stored in the table



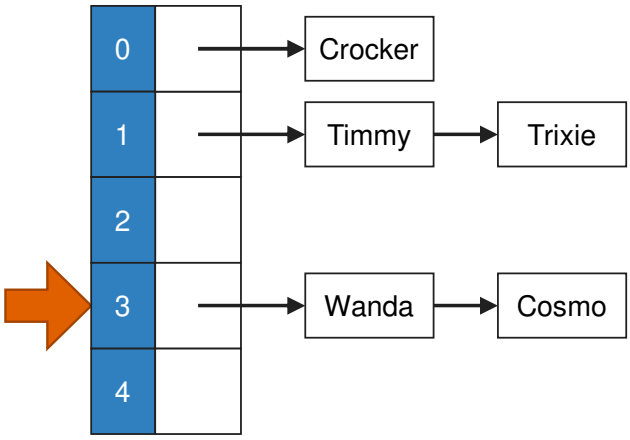
Open Hash Example

- Let's say with add "Trixie" and it hashes to 1
- This collides with "Timmy"
- In open hashing, Trixie is just added to the linked list (or tree)



Open Hash Example

- Likewise, we can add "Cosmo"
- Let's say he hashes to 3
- Once again, there is a collision with "Wanda", but we simply add the item to the linked list.



Open Hashing Benefits

1. Open hashing will not fill up the array and can grow *indefinitely*
2. Far faster access time than closed hashing
3. No clustering
4. Objects can be deleted

Let's Hash The Students!

Closed hashing

Let's Hash The Students!

- Let's add your names to a closed hash table
- First, let's figure out *how* we want to hash your names
- Then, I'll ask for 5 volunteers

Section 7: Characters in Name mod 6

	Name	Hash
1	Danny	5
2	Diego	5
3	Alex	4
4	Kelsey	0
5	Eli	3

Hash Table	
0	**Diego**
1	**Kelsey**
2	
3	Eli
4	Alex
5	Danny

Section 7: Birth Day of the Month mod 6

	Name	Hash
1	Christian	2
2	Danny	14 (2)
3	Tiffany	20 (2)
4	Josh	1
5	Bryce	3

Hash Table	
0	
1	Josh
2	Christian
3	**Danny**
4	**Tiffany**
5	**Byrce**

Section 8: Characters in Name mod 6

	Name	Hash
1	Jose	4
2	Alex	4
3	Brendon	7 (1)
4	Ki	2
5	William	7 (1)

Hash Table	
0	
1	Brendon
2	Ki
3	**William**
4	Jose
5	**Alex**

Section 8: Birth Day of the Month mod 6

	Name	Hash
1	Brendon	19 (1)
2	Alex	25 (1)
3	Jose	16 (4)
4	Alex (other)	18 (0)
5	Ki	15 (3)

Hash Table	
0	Alex (other)
1	Brendon
2	**Alex**
3	Ki
4	Jose
5	